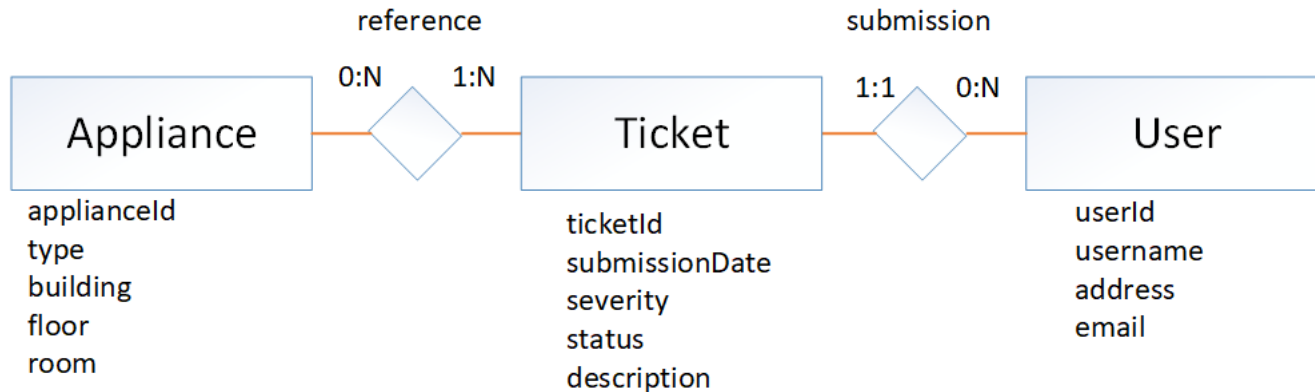


Databases 2

Exam on Jan 14 2022

JPA exercise solutions

Exercise on JPA



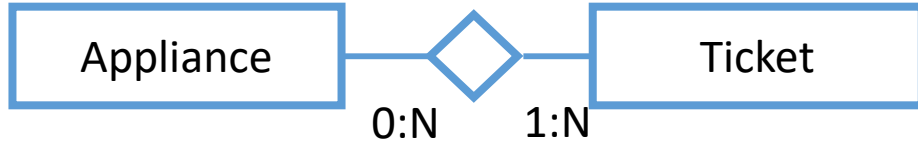
An application manages the tickets of a trouble ticketing service. The data are organized according to the following conceptual model.

The application permits the operator to select a user, open his/her list of tickets ordered by submission date descending, select a ticket and check the details of the appliances (one or more) the ticket refers to. Also the operator can select an appliance and check the tickets referring to it, open one such ticket and check its details and the details of the user who submitted it.

Tickets are in the order of millions, appliances are in the order of thousands and users are in the order of hundreds of thousands. Tickets are distributed uniformly across appliances and users.

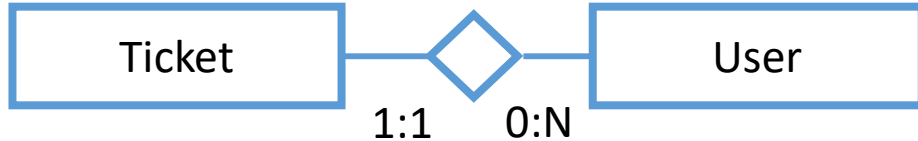
Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Relationship reference



- **Appliance → Ticket**
@ManyToMany
 - necessary to show the Tickets of a selected appliance
 - Owner = Ticket or Appliance
 - FetchType can be the default lazy mode (on average 1000 tickets per appliance)
 - No cascading is useful
- **Ticket → Appliance**
@ManyToMany
 - necessary to check the details of the appliances of a selected ticket
 - FetchType can be Eager (few appliances per ticket)
 - No cascading is useful

Relationship submission



- Ticket → User @ManyToOne
 - necessary to show the details of a ticket
 - Owner = Ticket
 - FetchType can be the default eager mode
 - Cascading not useful
- User → Ticket @OneToMany
 - necessary to show the tickets of a selected user
 - FetchType can be eager (10 tickets per user on average)
 - Remove can be cascaded

Entity Appliance

```
@Entity
public class Appliance{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int applianceId;
    private String type;
    private int building;
    private int floor;
    private int room;

    // no cascading. Cascade remove would also remove tickets related to
    // appliances that are still existing

    // lazy can be omitted as it is the default for collections
    @ManyToMany(mappedBy="appliances", fetch = FetchType.LAZY)
    private List<Ticket> tickets;

    //getters and setters...

}
```

Entity Ticket

```
@Entity
public class Ticket{

    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int ticketId;

    @Temporal(TemporalType.DATE)
    private Date submissionDate;
    private int severity; // string or enum equally ok
    private int status;   // string or enum equally ok
    private String description;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "reference",
        joinColumns = @JoinColumn(name = "ticketId"),
        inverseJoinColumns = @JoinColumn(name = "applianceId"))
    private List<Appliance> appliances; // owner of the relation

    @ManyToOne(fetch = FetchType.EAGER) // EAGER is default can be omitted
    @JoinColumn(name="user") // owner of the relation
    private User user;

    //getters and setters...
}
```

Entity User

```
@Entity
public class User{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int userId;
    private String username;
    private String address;
    private String email;

    @OneToMany(mappedBy="user", fetch = FetchType.EAGER,
               cascade=CascadeType.REMOVE)
    @OrderBy("submissionDate DESC")

    private List<Ticket> tickets;

    //getters and setters...
}
```